

Into to PF

Credit Hours: 3-0

Course Outline

Introduction to Programming Concepts, Algorithms and Flowcharts, Basics of Computer Systems and Programming Languages, Data Types and Variables, Input and Output Operations, Operators and Expressions, Control Structures (Selection and Iteration), Functions and Parameter Passing, Arrays (One-dimensional and Multi-dimensional), Strings and String Handling, Pointers and Memory Management, Structures and Unions, File Handling, Introduction to Object-Oriented Programming Concepts, Debugging and Error Handling, Basic Problem Solving and Program Design Techniques

16-Week Course Plan

Week 1

****Week 1****

Lecture 1: Introduction to Programming Concepts and Overview of the Course

Learning Objectives:

- Understand the basic concepts of programming
- Familiarize with the course outline and objectives

Concepts:

Programming is a skill that involves writing instructions that a computer can execute to solve problems or perform tasks. This course introduces the fundamental concepts of programming, including algorithms, flowcharts, and programming languages. We will explore the basics of computer systems and programming languages, which are essential for writing effective programs. By the end of this course, students will be able to design, write, and debug programs in a programming language of their choice.

Lecture 2: Basics of Computer Systems and Programming Languages

Learning Objectives:

- Understand the basic components of a computer system
- Familiarize with different programming languages

Concepts:

A computer system consists of hardware and software components. The hardware includes the central processing unit (CPU), memory, and input/output devices. The software includes the operating system, programming languages, and applications. Programming languages are used to write instructions that a computer can execute to solve problems or perform tasks. Some popular programming languages include Python, Java, C++, and C#.

Lecture 3: Basics of Computer Systems and Programming Languages (continued)

Learning Objectives:

- Understand the characteristics of different programming languages
- Familiarize with the syntax and semantics of a programming language

Concepts:

Programming languages vary in their syntax, semantics, and characteristics. Syntax refers to the rules of writing code, while semantics refers to the meaning of the code. Some programming languages are designed for specific purposes, such as web development or mobile app development. Others are general-purpose languages that can be used for a wide range of applications. By understanding the basics of computer systems and programming languages, students can make informed decisions about which language to use for a particular project.

****Note:**** Since the course has no lab hours for this week, there are no lab sessions scheduled.

Week 2

****Week 2: Basics of Computer Systems and Programming Languages****

Lecture 1: Computer Systems Overview (≈150 words)

Learning Objectives:

- Understand the basic components of a computer system
- Identify the different types of computer systems
- Recognize the role of software in computer systems

Concepts:

A computer system consists of hardware and software components. The hardware components include the central processing unit (CPU), memory, input/output devices, and storage devices. The software components include the operating system, applications, and firmware. There are two main types of computer systems: single-user systems and multi-user systems. Single-user systems are designed for personal use, while multi-user systems are designed for multiple users to access and share resources. Software plays a crucial role in computer systems, as it enables the hardware components to perform tasks and interact with users.

Lecture 2: Programming Languages Overview (≈150 words)

Learning Objectives:

- Understand the basic characteristics of programming languages
- Identify the different types of programming languages
- Recognize the importance of programming languages in computer systems

Concepts:

Programming languages are used to write instructions that a computer can execute. There are two main types of programming languages: low-level languages and high-level languages. Low-level languages, such as assembly languages, are machine-specific and require manual memory management. High-level languages, such as C and Java, are platform-independent and provide abstraction from hardware details. Programming languages can be further categorized into procedural languages, object-oriented languages, and functional languages. The choice of programming language depends on the problem to be solved, the desired level of abstraction, and the performance requirements.

Lecture 3: Programming Language Fundamentals (≈150 words)

Learning Objectives:

- Understand the basic syntax and semantics of programming languages
- Identify the different components of a programming language
- Recognize the importance of programming language standards

Concepts:

A programming language consists of syntax, semantics, and pragmatics. Syntax refers to the rules that govern the structure of programs, while semantics refers to the meaning of programs. Pragmatics refers to the use of programming languages in real-world applications. Programming languages have different components, including keywords, identifiers, literals, and operators. Programming language standards, such as those defined by the American National Standards Institute (ANSI), ensure that programs written in a particular language can be executed on different platforms. Understanding the fundamentals of programming languages is essential for writing efficient, portable, and maintainable code.

Week 3

****Week 3: Basics of Computer Systems and Programming Languages****

Lecture 1: Introduction to Computer Systems (30 minutes)

Computer System Components

A computer system consists of hardware and software components. Hardware components include the central processing unit (CPU), memory, input/output devices, and storage devices. Software components include the operating system, programming languages, and application software.

Learning Objectives

- Identify the hardware and software components of a computer system
- Explain the role of the CPU, memory, and input/output devices in a computer system

Concepts

- Hardware and software components of a computer system
- Central processing unit (CPU) and its role
- Memory and its types (RAM, ROM, etc.)
- Input/output devices and their types (keyboard, mouse, monitor, etc.)

Lecture 2: Programming Languages (45 minutes)

Types of Programming Languages

Programming languages can be classified into three categories: low-level, high-level, and assembly languages. Low-level languages are machine-dependent and require manual memory management. High-level languages are machine-independent and provide a higher level of abstraction.

Learning Objectives

- Identify the different types of programming languages
- Explain the characteristics of low-level, high-level, and assembly languages
- Compare the advantages and disadvantages of different programming languages

Concepts

- Types of programming languages (low-level, high-level, assembly)
- Characteristics of low-level and high-level languages
- Advantages and disadvantages of different programming languages

Lecture 3: Programming Language Paradigms (30 minutes)

Programming Language Paradigms

Programming language paradigms include imperative, object-oriented, and functional programming. Imperative programming focuses on procedures and loops, while object-oriented programming emphasizes objects and classes. Functional programming emphasizes the use of pure functions and immutability.

Learning Objectives

- Identify the different programming language paradigms
- Explain the characteristics of imperative, object-oriented, and functional programming
- Compare the advantages and disadvantages of different programming paradigms

Concepts

- Programming language paradigms (imperative, object-oriented, functional)
- Characteristics of imperative, object-oriented, and functional programming
- Advantages and disadvantages of different programming paradigms

Week 4

****Week 4: Control Structures (Iteration)****

Lecture 1: Introduction to Control Structures (Iteration) (≈75 words)

Introduction to Control Structures (Iteration)

Learning Objectives

- Understand the concept of control structures
- Learn about iteration using loops

Concepts

Control structures determine the flow of a program's execution. Iteration is a control structure that allows a program to repeat a set of instructions. Loops are the primary mechanism for iteration. They provide a way to execute a block of code repeatedly until a certain condition is met.

Lecture 2: Types of Loops (≈75 words)

Types of Loops

Learning Objectives

- Learn about different types of loops
- Understand the syntax of while loops and for loops

Concepts

There are two primary types of loops: while loops and for loops. While loops execute a block of code as long as a certain condition is true. For loops execute a block of code for a specified number of iterations.

Lecture 3: Loop Control Statements (≈75 words)

Loop Control Statements

Learning Objectives

- Learn about loop control statements
- Understand how to control the flow of loops

Concepts

Loop control statements are used to control the flow of loops. They include break statements, which exit a loop prematurely, and continue statements, which skip to the next iteration of a loop.

Week 5

****Week 5: Functions and Parameter Passing****

Lecture 1: Introduction to Functions (Theory) (60 minutes)

Functions in Programming

Functions are blocks of code that perform a specific task. They are reusable, help in code organization, and improve code modularity.

Learning Objectives

- * Understand the concept of functions in programming
- * Learn how to declare and use functions
- * Understand the importance of functions in code organization

Concepts

- * Function declaration and definition
- * Function signature and parameters
- * Function return types and values

Lecture 2: Function Parameters and Return Types (Theory) (45 minutes)

Function Parameters and Return Types

Functions can take parameters, which are values passed to the function when it is called. Functions can also return values to the calling code.

Learning Objectives

- * Understand how to declare function parameters
- * Learn about function return types and values
- * Understand the importance of parameter passing in function design

Concepts

- * Function parameter passing modes (value, reference, pointer)
- * Function return types (void, value, pointer)
- * Return statements and function exit points

Lecture 3: Using Functions in Programming (Theory) (45 minutes)

Using Functions in Programming

Functions are essential in programming for code organization, reusability, and modularity. They help in breaking down complex problems into smaller, manageable tasks.

Learning Objectives

- * Understand how to use functions in programming
- * Learn about function call stack and return addresses
- * Understand the importance of functions in program design

Concepts

- * Function call stack and return addresses
- * Function invocation and return
- * Function reuse and code modularity

No lab session for this week.

Week 6

****Week 6: Control Structures (Iteration)****

Lecture 1: Introduction to Iteration (≈75 words)

Iteration Control Structures

Learning Objectives:

- Understand the concept of iteration
- Learn about different types of iteration control structures
- Understand the importance of iteration in programming

Concepts:

Iteration is a process of repeating a set of instructions until a certain condition is met. There are two types of iteration control structures: for loops and while loops. For loops are used when the number of iterations is known in advance, while while loops are used when the number of iterations is not known.

Lecture 2: For Loops (≈75 words)**For Loops****Learning Objectives:**

- Understand the syntax of for loops
- Learn how to use for loops to iterate over arrays
- Understand the concept of loop counters

Concepts:

For loops are used to iterate over a range of values. The syntax of a for loop is: `for (init; condition; increment) { statement; }`. The init statement is executed once before the loop starts, the condition is checked before each iteration, and the increment statement is executed after each iteration.

Lecture 3: While Loops (≈75 words)**While Loops****Learning Objectives:**

- Understand the syntax of while loops
- Learn how to use while loops to iterate until a condition is met
- Understand the concept of loop termination

Concepts:

While loops are used to iterate until a certain condition is met. The syntax of a while loop is: `while (condition) { statement; }`. The condition is checked before each iteration, and the loop continues to execute as long as the condition is true.

Week 7

****Week 7: Functions and Parameter Passing****

Lecture 1: Introduction to Functions (Theory)**Functions: Building Blocks of Programs****Learning Objectives:**

- Understand the concept of functions in programming
- Learn to define and call functions

- Understand the benefits of using functions

Concepts:

Functions are reusable blocks of code that perform a specific task. They help in organizing and structuring programs, making them more modular and easier to maintain. Functions accept input parameters, perform calculations, and return output values.

Lecture 2: Function Arguments and Return Types (Theory)

Function Arguments and Return Types

Learning Objectives:

- Understand how to pass arguments to functions
- Learn about different types of function return values
- Understand the importance of function return types

Concepts:

Functions can accept input arguments, which are passed to the function when it is called. Functions can also return values, which are used by the calling program. The return type of a function specifies the type of value it returns.

Lecture 3: Function Overloading and Recursion (Theory)

Function Overloading and Recursion

Learning Objectives:

- Learn about function overloading and its benefits
- Understand the concept of recursion in programming
- Learn to use recursion to solve problems

Concepts:

Function overloading allows multiple functions with the same name but different parameters to be defined. Recursion is a programming technique where a function calls itself to solve a problem. Recursion is useful for solving problems that have a recursive structure.

Week 8

****Week 8: File Handling and Introduction to Object-Oriented Programming Concepts****

Lecture 1: File Handling (1.5 hours)

Introduction to File Handling

Files are used to store data permanently on a computer. File handling is an essential part of programming, as it allows data to be saved and retrieved for later use. There are two types of files: text files and binary files. Text files store data in a human-readable format, while binary files store data in a format that the computer can understand directly.

File Operations

The three main file operations are:

- * **Opening a file**: This involves creating a file object that allows you to interact with the file.
- * **Reading from a file**: This involves reading data from the file and storing it in a variable.
- * **Writing to a file**: This involves writing data to the file.

Example of File Handling in Programming

```
```python
file = open("example.txt", "w")
file.write("Hello, World!")
file.close()
```
```

Learning Objectives

- * Understand the concept of file handling in programming.
- * Learn how to perform file operations (opening, reading, and writing).
- * Understand the difference between text and binary files.

Concepts

- * File handling
- * Text files
- * Binary files
- * File operations (opening, reading, and writing)

Lecture 2: Introduction to Object-Oriented Programming Concepts (1.5 hours)

Introduction to Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. In OOP, programs are designed as a collection of objects that interact with each other.

Key Concepts in OOP

- * **Classes**: A class is a blueprint for creating objects. It defines the properties and behavior of an object.
- * **Objects**: An object is an instance of a class. It has its own set of attributes (data) and methods (functions).
- * **Inheritance**: Inheritance is a mechanism by which one class can inherit the properties and behavior of another class.
- * **Polymorphism**: Polymorphism is the ability of an object to take on multiple forms.

Example of OOP in Programming

```
```python
class Animal:
 def __init__(self, name):
 self.name = name
 def speak(self):
 print("Animal speaks")
class Dog(Animal):
 def speak(self):
 print("Dog barks")
my_dog = Dog("Buddy")
my_dog.speak()
```
```

Learning Objectives

- * Understand the concept of Object-Oriented Programming.
- * Learn about key OOP concepts (classes, objects, inheritance, and polymorphism).
- * Understand how to implement OOP in programming.

Concepts

- * Object-Oriented Programming
- * Classes
- * Objects
- * Inheritance
- * Polymorphism

Lecture 3: Introduction to Object-Oriented Programming Concepts (1 hour)

Review of OOP Concepts

Review the key concepts in OOP, including classes, objects, inheritance, and polymorphism.

Real-World Applications of OOP

Discuss how OOP is used in real-world applications, such as game development, web development, and mobile app development.

Best Practices for Implementing OOP

Discuss best practices for implementing OOP, including encapsulation, abstraction, and modularity.

Example of OOP in a Real-World Application

```
```python
class BankAccount:
 def __init__(self, account_number, balance):
 self.account_number = account_number
 self.balance = balance
 def deposit(self, amount):
 self.balance += amount
 def withdraw(self, amount):
 if amount > self.balance:
 print("Insufficient funds")
 else:
 self.balance -= amount
account = BankAccount("123456789", 1000)
account.deposit(500)
print(account.balance)
```
```

Learning Objectives

- * Review key OOP concepts (classes, objects, inheritance, and polymorphism).
- * Understand how OOP is used in real-world applications.
- * Learn best practices for implementing OOP.

Concepts

- * OOP concepts (classes, objects, inheritance, and polymorphism)
- * Real-world applications of OOP
- * Best practices for implementing OOP

Week 9

****Week 9: Pointers and Memory Management****

Lecture 1: Introduction to Pointers (~75 words)

Pointers and Memory Management Basics

Learning Objectives:

- * Understand the concept of pointers in programming
- * Learn to declare and initialize pointers

Concepts:

A pointer is a variable that stores the memory address of another variable. In C, a pointer is declared using the asterisk symbol (*). For example, `int *ptr;` declares a pointer to an integer. Pointers are used to access and manipulate memory locations directly.

Lecture 2: Pointer Arithmetic and Operations (≈75 words)

Pointer Arithmetic and Operations

Learning Objectives:

- * Understand pointer arithmetic operations (e.g., ++, --, +=, -=)
- * Learn to perform pointer comparisons and assignments

Concepts:

Pointer arithmetic involves performing operations on pointers to access different memory locations. For example, `ptr++` increments the pointer to point to the next memory location. Pointers can also be compared using relational operators (e.g., `==`, `!=`).

Lecture 3: Memory Management with Pointers (≈75 words)

Memory Management with Pointers

Learning Objectives:

- * Understand how to dynamically allocate memory using pointers
- * Learn to deallocate memory using pointers

Concepts:

Dynamic memory allocation involves using functions like `malloc()` to allocate memory at runtime. Pointers are used to store the memory address returned by `malloc()`. Memory can be deallocated using `free()`. Proper memory management is essential to prevent memory leaks and dangling pointers.

Week 10

****Week 10: Introduction to Object-Oriented Programming Concepts****

Lecture 1: Introduction to OOP (≈75 words)

Introduction to Object-Oriented Programming

Learning Objectives

- Define object-oriented programming (OOP) concepts.
- Identify the fundamental principles of OOP.

Concepts

Object-Oriented Programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. In OOP, programs are designed as a collection of objects that interact with each other. The key principles of OOP include encapsulation, inheritance, polymorphism, and abstraction.

Lecture 2: Classes and Objects (≈75 words)

Classes and Objects

Learning Objectives

- Define classes and objects in OOP.
- Explain the relationship between classes and objects.

Concepts

In OOP, a class is a blueprint or a template that defines the properties and behavior of an object. An object is an instance of a class and has its own set of attributes (data) and methods (functions). Classes define the structure of objects, while objects represent specific instances of that structure.

Lecture 3: Inheritance (≈75 words)

Inheritance

Learning Objectives

- Define inheritance in OOP.
- Explain the types of inheritance.

Concepts

Inheritance is a mechanism in OOP that allows one class to inherit the properties and behavior of another class. The inheriting class is called the subclass or derived class, while the class being inherited from is called the superclass or base class. There are two types of inheritance: single inheritance and multiple inheritance.

Lecture 4: Polymorphism (≈75 words)

Polymorphism

Learning Objectives

- Define polymorphism in OOP.
- Explain the types of polymorphism.

Concepts

Polymorphism is a fundamental concept in OOP that allows objects of different classes to be treated as objects of a common superclass. There are two types of polymorphism: method overriding and method overloading. Method overriding occurs when a subclass provides a specific implementation of a method that is already defined in its superclass. Method overloading occurs

when multiple methods with the same name but different parameters are defined in a class.

Week 11

****Week 11:****

Lecture 11.1: Introduction to Object-Oriented Programming (OOP) Concepts (≈75 words)

Object-Oriented Programming

Learning Objectives

- * Define object-oriented programming and its key concepts.
- * Identify the benefits of OOP.

Concepts

Object-oriented programming (OOP) is a programming paradigm that revolves around the concept of objects and classes. OOP focuses on creating reusable code through encapsulation, inheritance, and polymorphism. Key benefits of OOP include modularity, flexibility, and easier maintenance.

Lecture 11.2: Classes and Objects (≈75 words)

Classes and Objects

Learning Objectives

- * Define classes and objects in OOP.
- * Identify the characteristics of classes and objects.

Concepts

In OOP, a class is a blueprint or template for creating objects. An object is an instance of a class and has its own set of attributes (data) and methods (behaviors). Classes can inherit properties and behaviors from other classes, and objects can interact with each other through methods.

Lecture 11.3: Inheritance and Polymorphism (≈75 words)

Inheritance and Polymorphism

Learning Objectives

- * Explain inheritance and polymorphism in OOP.
- * Identify the benefits of inheritance and polymorphism.

Concepts

Inheritance allows a class to inherit properties and behaviors from another class. Polymorphism enables objects of different classes to be treated as objects of a common superclass. Inheritance promotes code reuse and reduces duplication, while polymorphism enhances flexibility and improves code maintainability.

Week 12

****Week 12****

****Credit Hours:** 3-0 (3 theory, 0 lab)**

Lecture 1: Structures and Unions (1.5 hours)

Learning Objectives

- * Understand the concept of structures and unions in programming.
- * Learn how to define and use structures and unions in a program.
- * Understand the difference between structures and unions.

Concepts

A structure is a collection of variables of different data types that can be stored and manipulated as a single unit. A union is a collection of variables of different data types that occupy the same memory location. Structures and unions are used to group related variables together and provide a way to access and manipulate multiple variables at once. In programming, structures and unions are used to represent complex data types and to improve code readability and maintainability.

Lecture 2: Structures and Unions (continued) (1.5 hours)

Learning Objectives

- * Learn how to access and manipulate elements of a structure or union.
- * Understand how to use pointers with structures and unions.
- * Learn how to define and use nested structures and unions.

Concepts

Elements of a structure or union can be accessed using the dot (.) operator. Pointers can be used with structures and unions to access and manipulate their elements. Nested structures and unions are used to represent complex data types and to improve code readability and maintainability. To define a nested structure or union, we use the struct or union keyword followed by the name of the nested structure or union.

Week 13

****Week 13: Object-Oriented Programming Concepts (3 lectures, 0 lab)****

Lecture 1: Classes and Objects

Classes and Objects

In object-oriented programming, a class is a blueprint or a template that defines the properties and behaviors of an object. An object is an instance of a class, which contains its own set of attributes (data) and methods (functions).

Learning Objectives:

- * Define classes and objects in object-oriented programming.
- * Identify the characteristics of classes and objects.

Concepts:

- * Classes and objects in OOP
- * Attributes and methods of a class
- * Instantiation of objects from classes

Lecture 2: Inheritance and Polymorphism**Inheritance**

Inheritance is a mechanism in which one class can inherit the properties and behaviors of another class. The inheriting class is called the subclass or derived class, and the class being inherited from is called the superclass or base class.

Polymorphism

Polymorphism is the ability of an object to take on multiple forms. This can be achieved through method overloading or method overriding.

Learning Objectives:

- * Define inheritance and polymorphism in object-oriented programming.
- * Explain the concept of inheritance and polymorphism.

Concepts:

- * Inheritance in OOP
- * Polymorphism in OOP
- * Method overloading and method overriding

Lecture 3: Encapsulation and Abstraction**Encapsulation**

Encapsulation is the concept of hiding the internal details of an object from the outside world and only exposing the necessary information through public methods.

Abstraction

Abstraction is the concept of showing only the essential features of an object while hiding the non-essential details.

Learning Objectives:

- * Define encapsulation and abstraction in object-oriented programming.
- * Explain the importance of encapsulation and abstraction.

Concepts:

- * Encapsulation in OOP
- * Abstraction in OOP

- * Importance of encapsulation and abstraction

Week 14

****Week 14: File Handling****

Lecture 1: Introduction to File Handling (≈150 words)

Learning Objectives

- * Understand the concept of file handling in programming
- * Learn to create, read, and write files using programming languages

Concepts

File handling is a crucial aspect of programming that enables you to store and retrieve data from files. Files can be used to store user input, program output, or any other type of data. Programming languages provide various functions and methods to create, read, and write files. In this lecture, we will discuss the basics of file handling, including file types, file operations, and file modes. We will also explore how to use programming languages to create, read, and write files.

Lecture 2: Reading and Writing Files (≈150 words)

Learning Objectives

- * Learn to read and write files using programming languages
- * Understand the different file modes and operations

Concepts

In this lecture, we will delve deeper into the concepts of reading and writing files. We will discuss the different file modes, such as text mode and binary mode, and how to use them to read and write files. We will also explore various file operations, including opening, closing, and seeking files. Additionally, we will learn how to handle file exceptions and errors.

Lecture 3: File Handling with Programming Languages (≈150 words)

Learning Objectives

- * Learn to use programming languages to perform file handling operations
- * Understand how to handle file exceptions and errors

Concepts

In this lecture, we will explore how to use programming languages to perform file handling operations. We will discuss how to create, read, and write files using various programming languages, including Python, Java, and C++. We will also learn how to handle file exceptions and errors, including how to use try-catch blocks and error handling mechanisms. By the end of this lecture, you will be able to write programs that can read and write files using programming languages.

Week 15

****Week 15: Pointers and Memory Management****

Lecture 1: Introduction to Pointers (≈150 words)

Pointers and Memory Management

Learning Objectives

- * Understand the concept of pointers in programming
- * Learn to declare and initialize pointers
- * Understand the memory allocation process

Concepts

A pointer is a variable that stores the memory address of another variable. It's used to access and manipulate data in memory. Pointers are declared using the asterisk symbol (*). For example, `int *ptr;` declares a pointer to an integer. Pointers are initialized by assigning the address of a variable to them. The memory allocation process involves reserving space in memory for variables and data structures.

Lecture 2: Pointer Arithmetic and Operations (≈150 words)

Pointer Arithmetic and Operations

Learning Objectives

- * Understand pointer arithmetic operations
- * Learn to perform pointer operations using the increment and decrement operators
- * Understand how to store and retrieve values using pointers

Concepts

Pointer arithmetic involves performing operations on pointers to access different memory locations. The increment (`++`) and decrement (`--`) operators can be used to move the pointer to the next or previous memory location. For example, `ptr++` increments the pointer to point to the next memory location. Pointers can be used to store and retrieve values from memory using the dereference operator (*).

Lecture 3: Dynamic Memory Allocation and Deallocation (≈150 words)

Dynamic Memory Allocation and Deallocation

Learning Objectives

- * Understand the concept of dynamic memory allocation
- * Learn to allocate and deallocate memory using functions like `malloc()` and `free()`
- * Understand the importance of memory deallocation

Concepts

Dynamic memory allocation involves allocating memory at runtime using functions like `malloc()` and `calloc()`. The `malloc()` function returns a pointer to the allocated memory, which can be used to store data. Memory deallocation is essential to free up memory occupied by dynamically

allocated variables. The `free()` function is used to deallocate memory.

Week 16

****Week 16: Basic Problem Solving and Program Design Techniques****

Lecture 1: Introduction to Problem Solving (≈75 words)

Problem Solving and Program Design

Learning Objectives:

- Understand the importance of problem-solving in programming
- Define problem-solving and program design
- Identify the steps involved in problem-solving

Concepts:

Problem-solving is a critical skill for programmers. It involves identifying, analyzing, and solving problems. Program design is a systematic approach to problem-solving. The steps involved in problem-solving include: (1) defining the problem, (2) gathering information, (3) developing a plan, (4) implementing the plan, and (5) testing and evaluating the solution.

Lecture 2: Program Design Techniques (≈75 words)

Program Design Techniques

Learning Objectives:

- Understand different program design techniques
- Identify the advantages of each technique
- Apply program design techniques to real-world problems

Concepts:

Program design techniques include top-down, bottom-up, and structured design. Top-down design involves breaking down the problem into smaller sub-problems. Bottom-up design involves developing a solution for each sub-problem and then integrating them. Structured design involves using a modular approach to develop a program. Each technique has its advantages and disadvantages, and the choice of technique depends on the problem and the programmer's preference.

Lecture 3: Program Evaluation and Testing (≈75 words)

Program Evaluation and Testing

Learning Objectives:

- Understand the importance of program evaluation and testing
- Identify different testing techniques
- Apply program evaluation and testing to improve program quality

Concepts:

Program evaluation and testing are critical steps in the problem-solving process. They help ensure that the program meets the requirements and produces the desired output. Testing techniques include unit testing, integration testing, and system testing. Program evaluation involves analyzing the program's performance, efficiency, and reliability. By applying program evaluation and testing techniques, programmers can improve the quality of their programs and ensure that they meet the requirements.